

End-to-End Automated Sales ETL Pipeline & Business Intelligence Dashboard using SSIS, SQL Server, and Power BI

Developed by: Ahmed Samy

Data Analytics Engineer Internship at EJADA

Under the supervision of:
Eng. Ahmed Abdelhakeem
Eng. Abu Bakr Makhyoun

Why Ejada is a Premier Destination for Aspiring Data Professionals ?

Ejada Systems Ltd. is one of the largest and most respected IT services and solutions companies in the MENA region.

With 20+ years of experience, Ejada delivers digital transformation projects for government, healthcare, telecom, and finance sectors across the Kingdom of Saudi Arabia and beyond.

Internship Highlights:

- Structured Learning
- The internship is designed with a clear technical path, introducing us to enterprise-level data lifecycle management.
- Real-World Projects.
- Valuable guidance from experienced professionals who provided practical feedback, technical direction, and shared best practices in tools like SSIS, SQL Server.

Learn more: www.ejada.com



The Problem: Why OLTP Systems Aren't Built for Analytics

- OLTP (Online Transaction Processing) systems are optimized for real-time business operations, not for analytics or reporting.
- Business users face several challenges when analyzing data from OLTP systems:
- Complex and normalized table structures
- No pre-aggregated views for fast reporting
- Poor query performance on large datasets

OLTP systems lack native support for:

1. Historical data tracking (e.g., Slowly Changing Dimensions)
2. Time-based aggregation and trend analysis
3. Incremental data extraction and loading
4. As a result, generating reports directly from OLTP leads to performance issues, data inconsistency, and limited insights.

To make data useful for decision-making, we needed to reshape and automate how it's stored and analyzed.

Understanding the Source Data



Adventure Works Cycles is a multinational company for bikes manufacturing, the company continues to grow. They demand robust Analytics Solution to store and analyse their sales for better monitoring and decision-making.

The project is based on a transactional OLTP database that simulates a retail sales system.

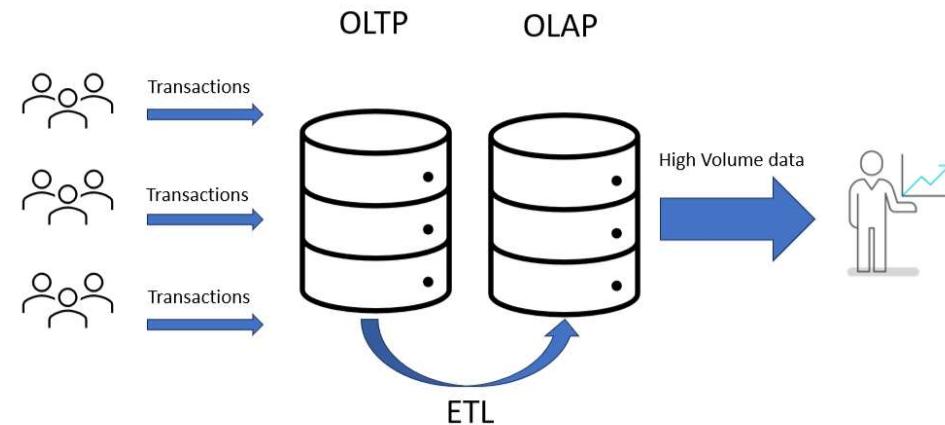
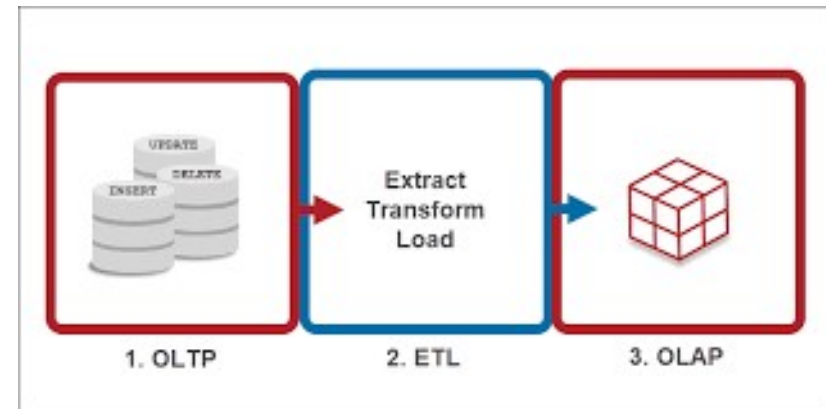
- The data includes several key entities:
- Customers and their addresses
- Products and product categories
- Sales orders and their details (quantities, discounts, prices)
- Order dates and shipping dates
- The data structure is normalized and distributed across multiple relational tables in the SalesLT schema.
- The raw data lacks analytics-ready features such as dimensions, surrogate keys, and pre-aggregated facts.

Project Goal

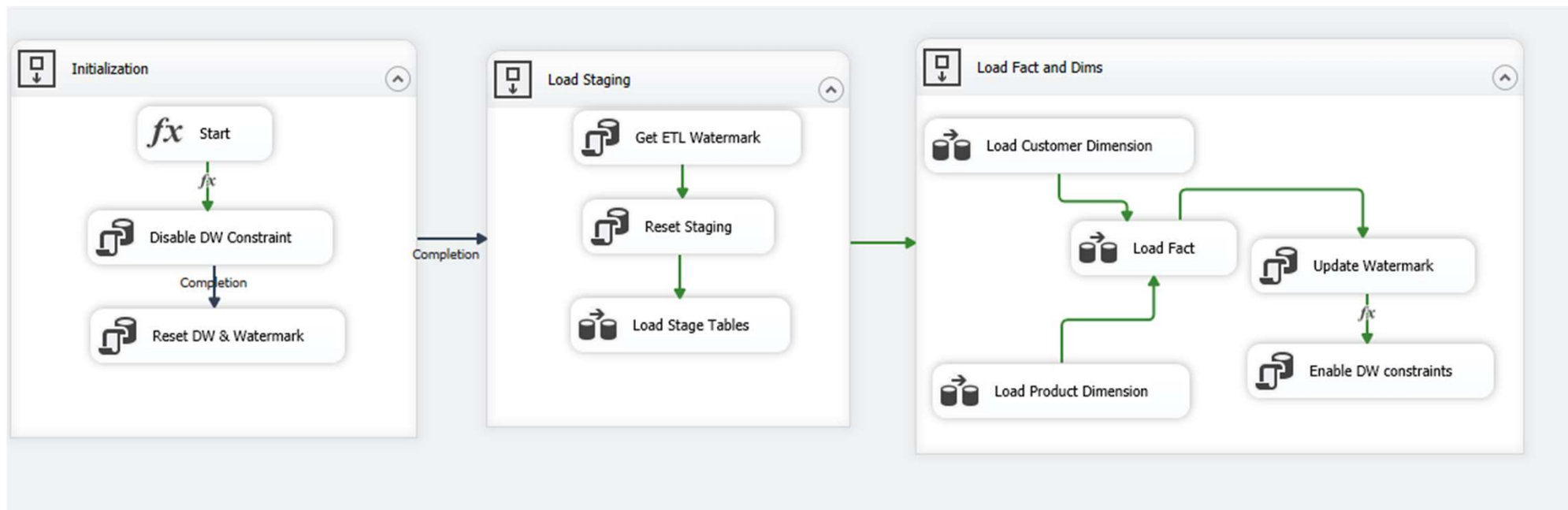
The primary goal of the project is to enable fast, consistent, and insightful analysis on sales data, which is difficult to achieve directly using an OLTP system due to its normalized structure and performance limitations.

To accomplish this, we aimed to:

- Build a mature and predictable data model by transforming raw OLTP data into a well-structured OLAP format.
- Support analytical needs such as trend analysis, KPIs, and dimensional slicing (e.g., by customer, product, category, date).
- Ensure scalable performance for reporting tools like Power BI by loading cleaned and enriched data into a dimensional model.
- Implement both Initial and Incremental loading strategies to handle large historical datasets and ongoing data changes efficiently.



ETL Architecture & Execution Flow



ETL Architecture & Execution Flow

Columnstore Index in Fact Table

To improve performance and reduce storage space, a Clustered Columnstore Index was created on the FactSales table.

This index allows faster aggregation and scanning for large volumes of data, which is essential for analytical workloads.

Why Staging is Important

- Acts as a buffer between OLTP and DW.
- Enables controlled transformation, cleansing, and debugging.
- Helps isolate extraction and loading phases.

Column Store Index										
Row 1	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Row 2	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Row 3	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Row 4	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Row 5	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Row 6	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Row 7	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Row 8	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
.....	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Row n	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
	Page 1	Page 2	Page 3	Page 4	Page 5	Page 6	Page 7	Page 8	Page 9	Page 10

ETL Architecture & Execution Flow

1. Initial Load Mechanism

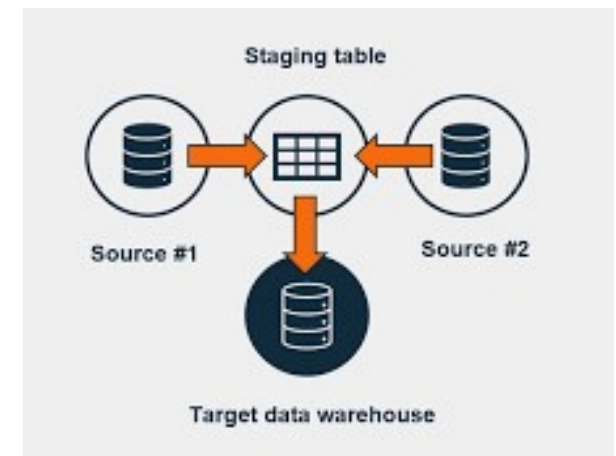
- Disable DW constraints to avoid foreign key violations and allow clean data reload.
- Reset the Data Warehouse (clear previous data and watermark).
- After the load is complete, constraints are re-enabled (This step only runs when the InitialLoad parameter is set to True.)

2. Staging Load

- Retrieve the latest watermark value (LastSuccessfulRun).
- Extract only new or updated data from OLTP using ModifiedDate.
- Reset staging tables and load fresh data incrementally.

3. DW Load

- Load dimension tables (DimCustomer, DimProduct) using Slowly Changing Dimension (SCD) logic.
- Lookup and generate surrogate keys.
- Load fact table (FactSales) using valid surrogate keys only.
- Update the watermark with the current job execution time.

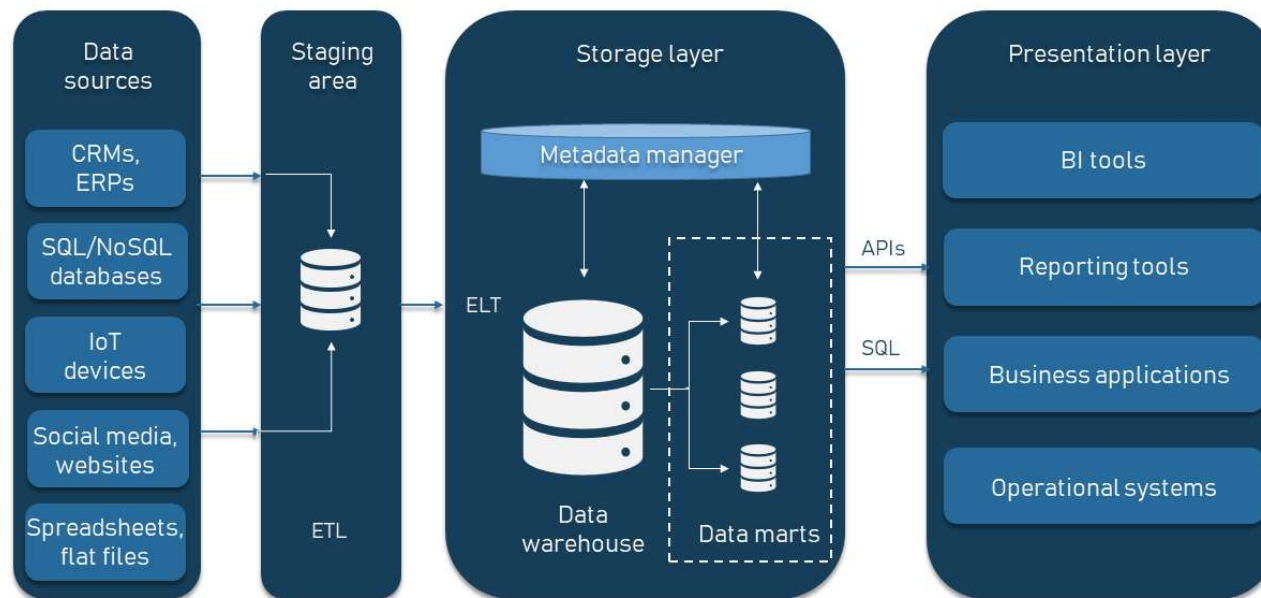


Watermark logic ensures incremental loads only.

ETL flow is fully controlled using SSIS control flow and dynamic parameters.

Data Warehouse Architecture

ENTERPRISE DATA WAREHOUSE COMPONENTS



OLTP Source

Raw operational data from SalesLT (Customers, Products, Orders)



Staging Layer

Temporary storage for new/updated data using Watermark logic



ETL Layer (SSIS)

- Merge & transform data
- Handle SCDs
- Resolve surrogate keys



DW Layer

- Dimensions: Customer (Type 1), Product (Type 2), Date
- Fact: FactSales with columnstore index



Metadata Layer

Watermark table to track last ETL run for incremental loading



Use Cases: Initial Load vs Incremental Load

Shared Implementation Logic

- Both Initial Load and Incremental Load share the same ETL logic:
- Use the same SSIS package and Data Flow design.
- Extract from OLTP → Transform → Load to DW.
- Use staging layer and watermarking.
- Controlled by a Parameter: IsInitialLoad
- A package-level Boolean parameter that changes the behavior based on load type.

Initial Load

Used for:

- First-time load or full reload of the DW.
- Re-enable constraints after load completes.
- Incremental Load
- Used for:
- Daily or periodic updates.

Logic:

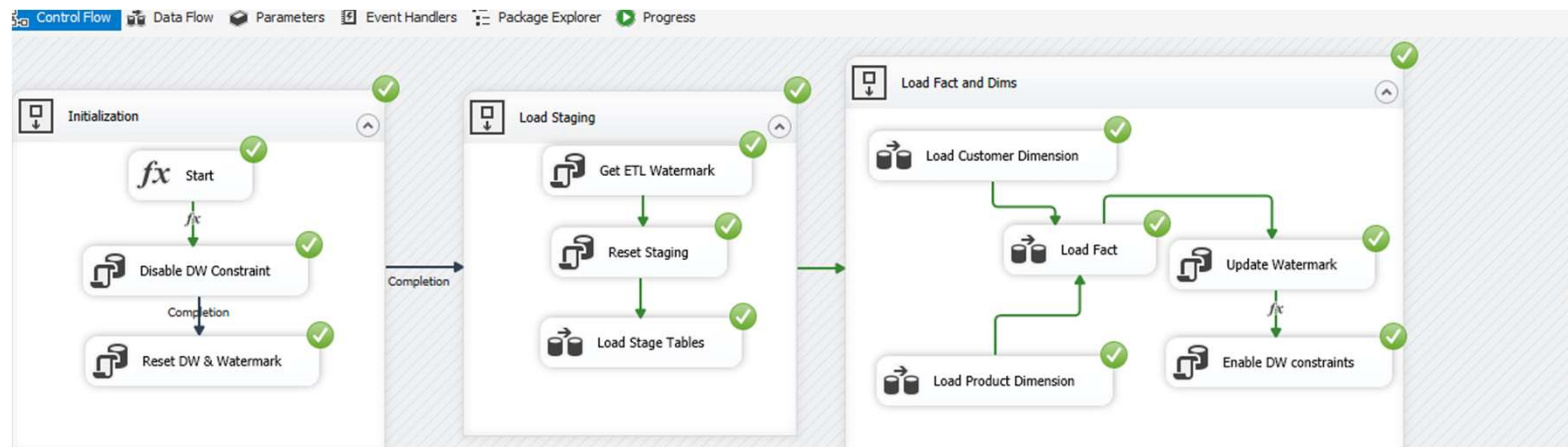
- Use the latest Watermark value to filter changed records (ModifiedDate).
- Append only new or updated data to staging and DW.
- No need to disable constraints or truncate tables.

Initial Load Results

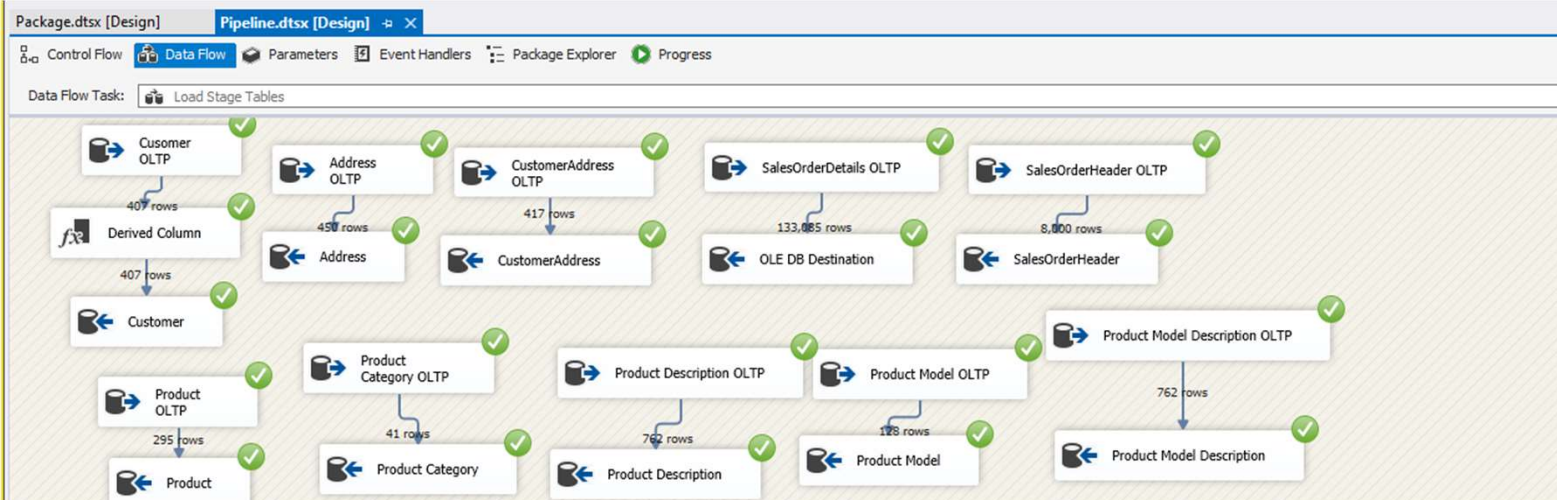
InitialLoad=True

	Data type	Value
INITIALLOAD	Boolean	True
Language	String	en

Package	CurrentJobTime	Package	Data type	Value
	Package		DateTime	12/30/2022 12:00 PM
	Package		DateTime	1/1/1900 12:00 PM



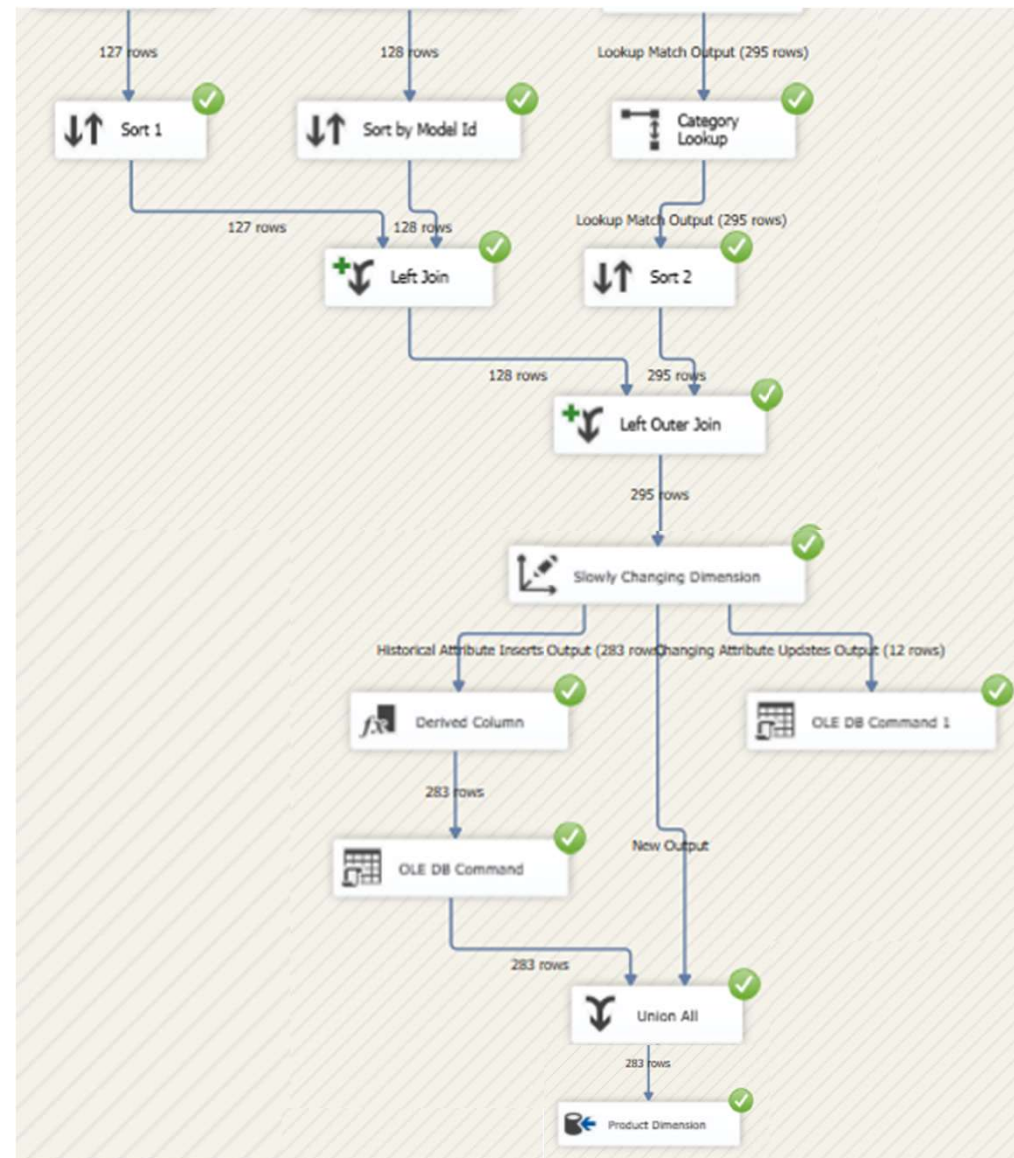
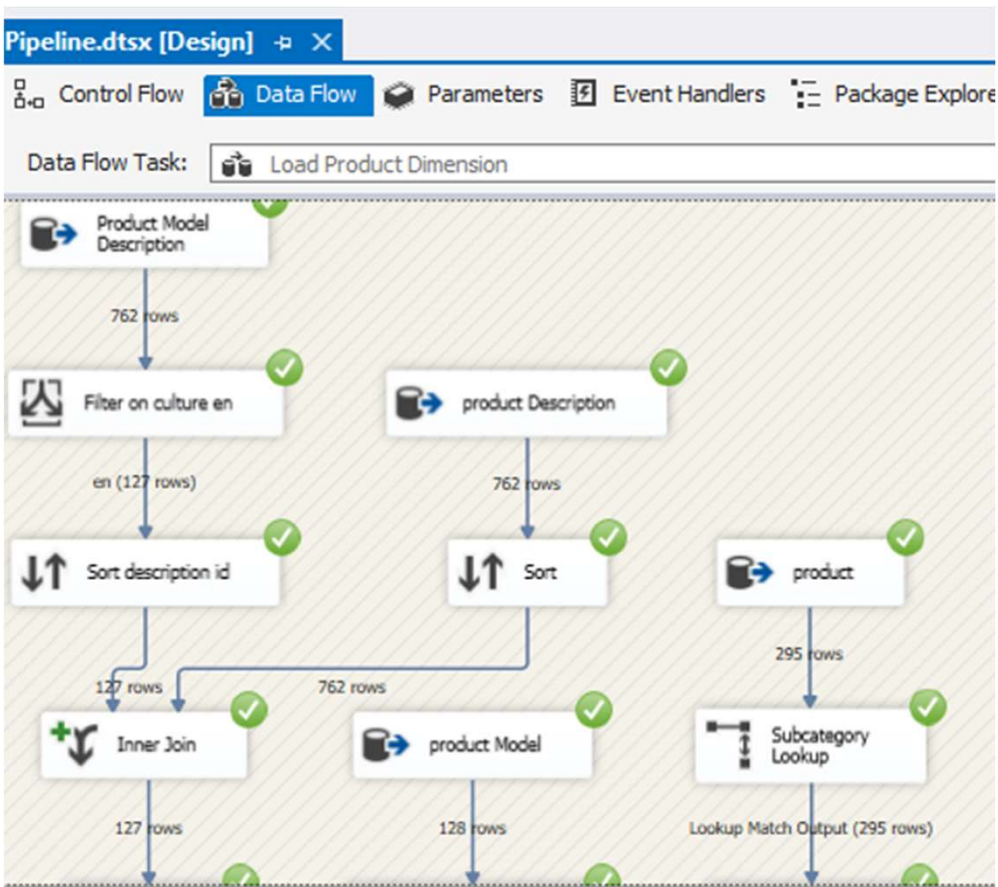
```
SELECT TOP (1000) [LastSuccessfulRun]
FROM [AdventureWorks-DW].[dbo].[Watermar
```



Results Messages

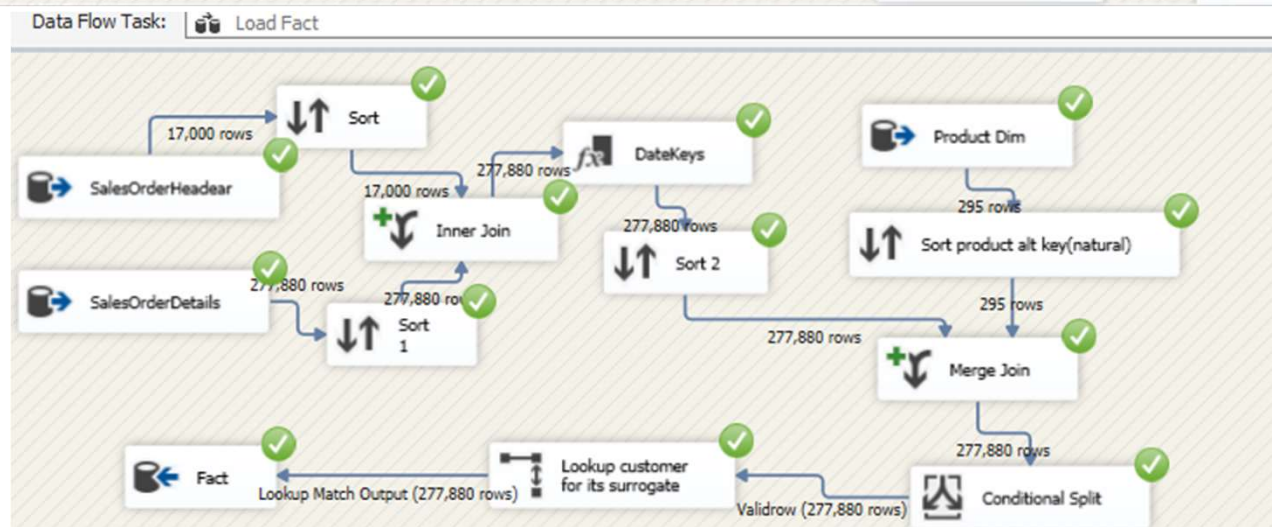
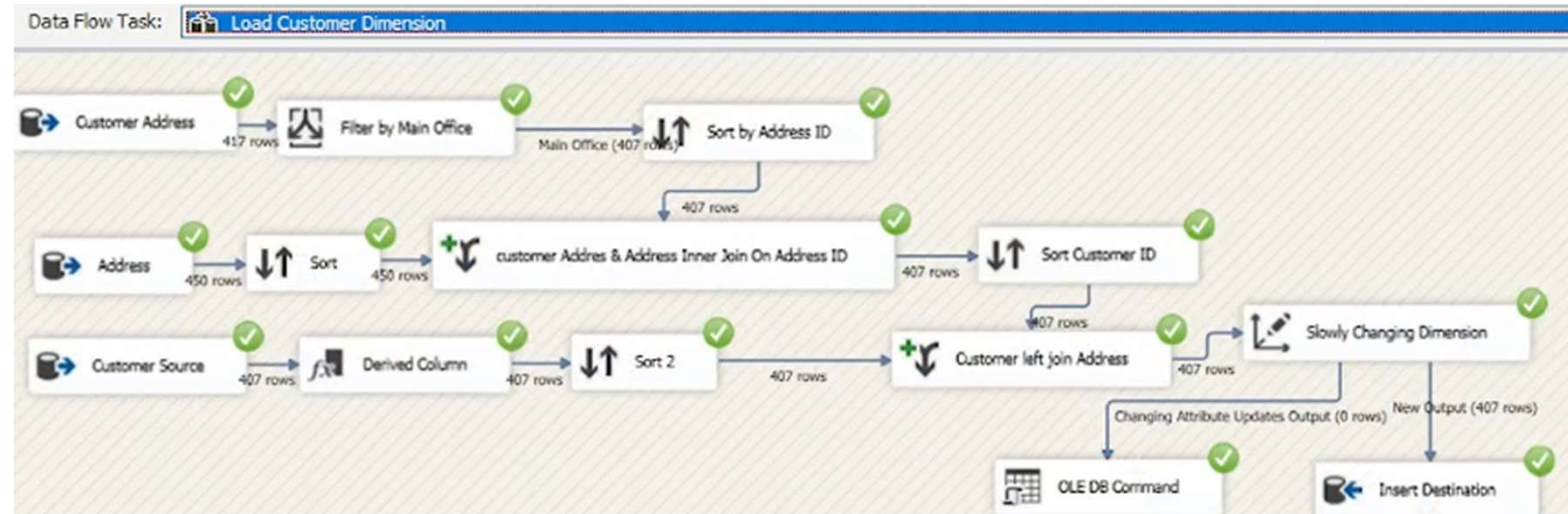
LastSuccessfulRun
2022-12-30 12:00:00.000

Product Dimension



Initial Load Results

Customer Dimension and Fact

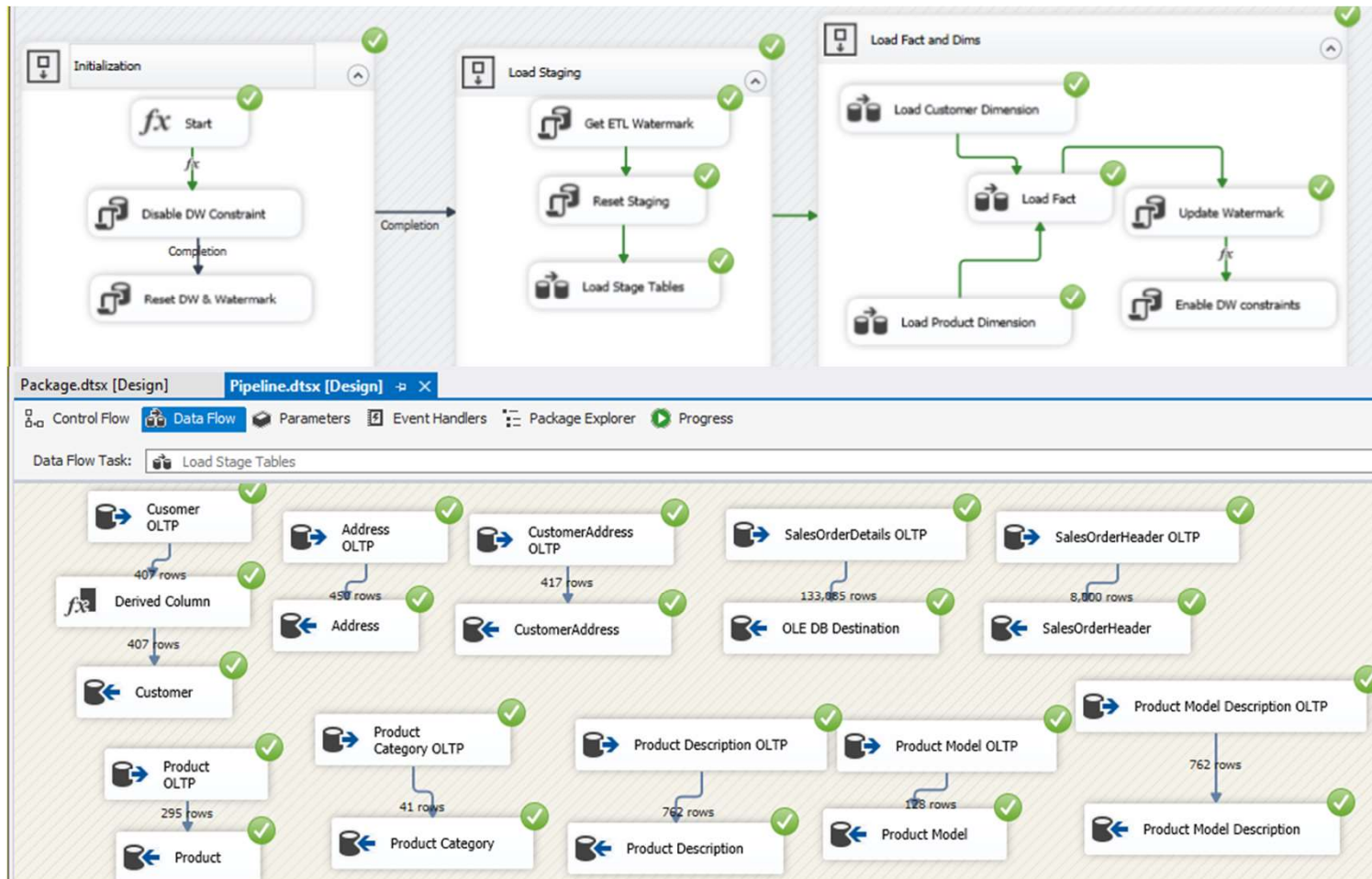


Incremental Load Results

InitialLoad=False

Name	Data type	Value	Sensitive	Required
INITIALLOAD	Boolean	False	False	False
Language	String	en	False	False

Name	Scope	Data type	Value
CurrentJobTi...	Package	DateTime	12/30/2026 12:00 ...

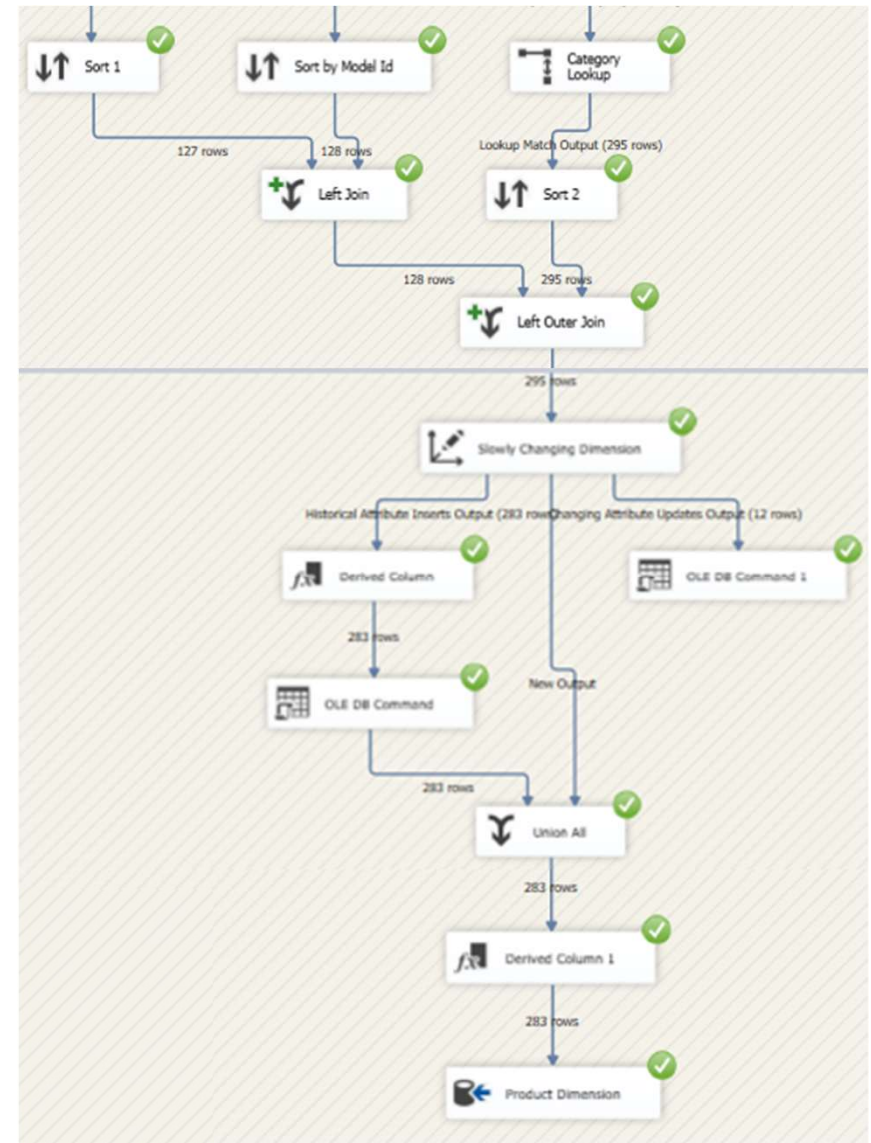
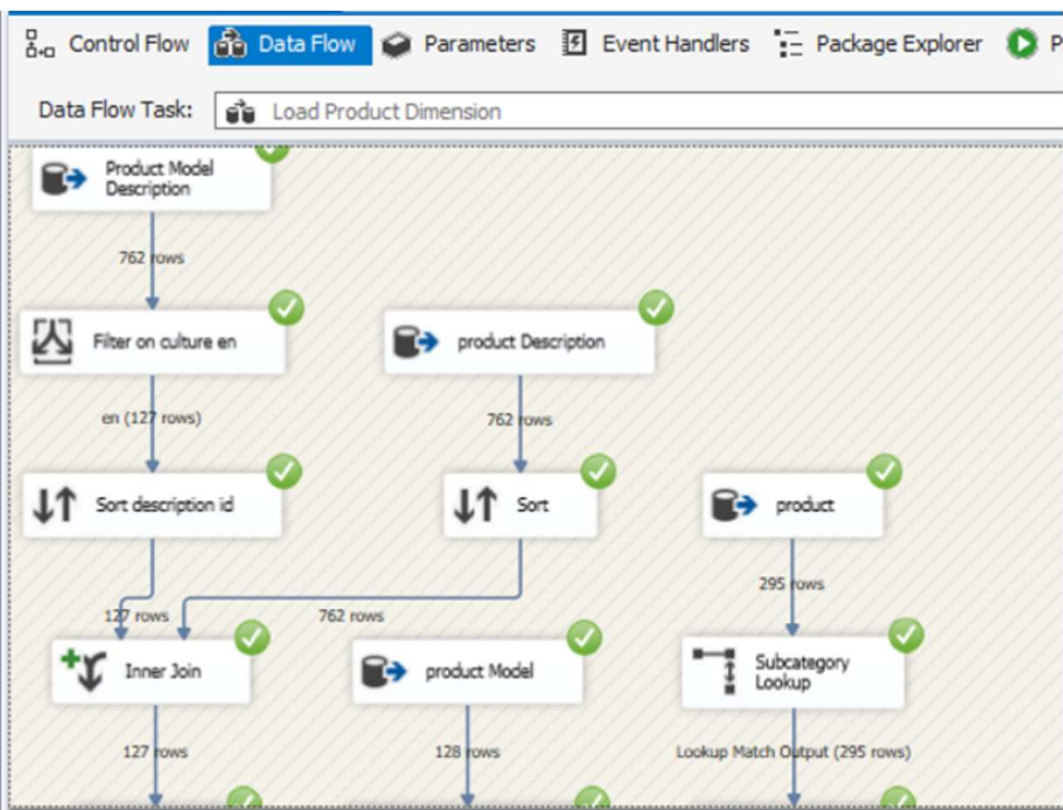


```
SELECT TOP (1000) [LastSuccessfulRun]
FROM [AdventureWorks-DW].[dbo].[Watermark]
```

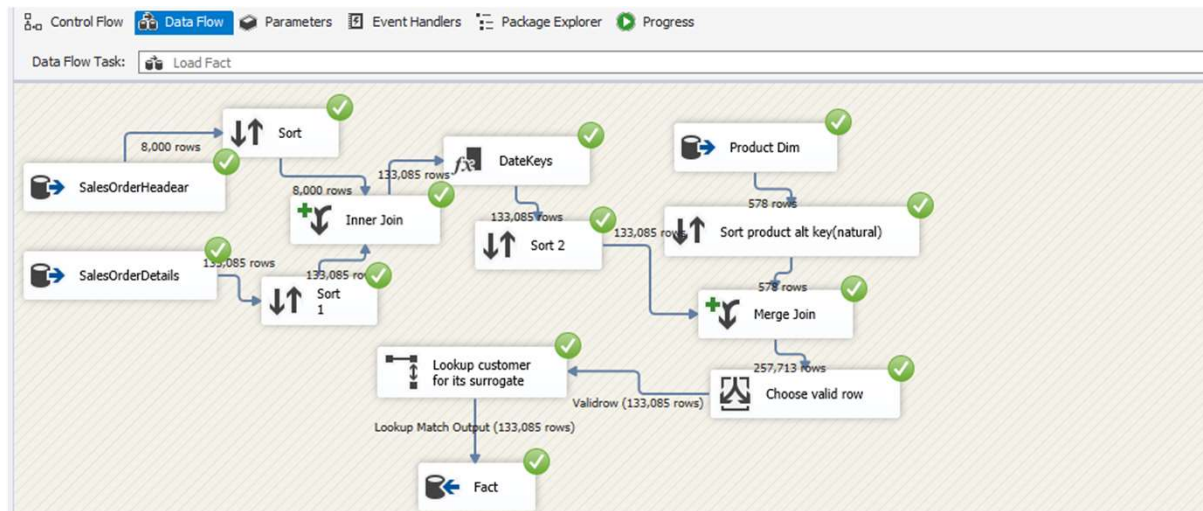
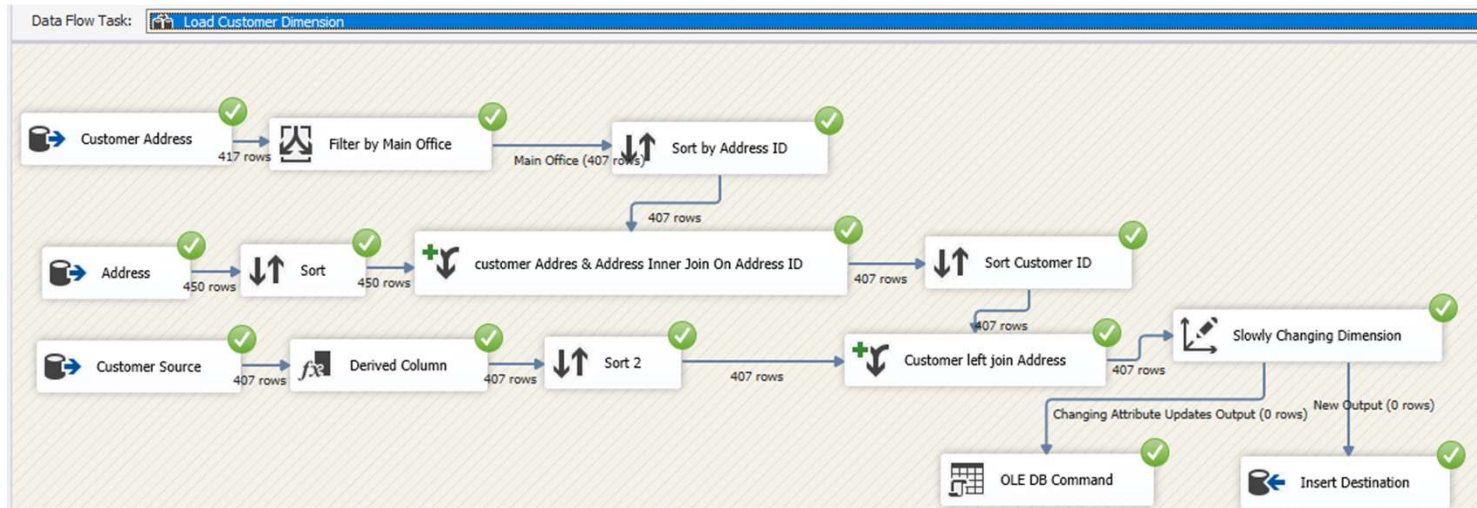
results Messages
Last Successful Run
2026-12-30 12:00:00.000

Incremental Load Results

Product Dimension



Customer Dimension and Fact



Results

OLTP vs OLAP Queries

OLTP

OLTP Query.sql - (local)\OLTP.AdventureWorks-OLTP (DESKTOP-LEIQFVB\hp (53)) - Microsoft SQL Server Management Studio

```
USE [AdventureWorks-OLTP];
GO

SET STATISTICS TIME ON;

WITH OrderLines AS (
    SELECT
```

Year	Month	Country	Category	OrderCount	TotalSales	GrossSales	AvgDiscountPerUnit	AvgDaysToShip	TaxRatio	FreightRatio
2020	1	Canada	Accessories	32	1949131.6500	23677.3900000	0.100000	8	0.069221	0.000467
2020	1	Canada	Bikes	69	3180100.7985	2842903.9754000	0.100000	8	0.069727	0.000615
2020	1	Canada	Clothing	57	2829145.2912	70295.6100000	0.104980	8	0.069936	0.000566
2020	1	Canada	Components	54	2793489.4569	358905.7300000	0.100000	8	0.069999	0.000579
2020	1	United Kingdom	Accessories	8	584493.8910	4390.4600000	0.100000	8	0.076375	0.000446
2020	1	United Kingdom	Bikes	25	1122157.4419	971112.1327000	0.100000	8	0.073149	0.000653
2020	1	United Kingdom	Clothing	18	879673.6899	18486.4900000	0.100000	8	0.074902	0.000630
2020	1	United Kingdom	Components	20	1033178.9728	163246.4250000	0.100000	8	0.072475	0.000558
2020	1	United States	Accessories	68	4250450.8715	55974.0900000	0.100000	8	0.066130	0.000455
2020	1	United States	Bikes	181	7525602.5123	6494776.4955000	0.100000	8	0.068157	0.000654
2020	1	United States	Clothing	131	6404687.0609	164990.5000000	0.104225	8	0.067914	0.000556
2020	1	United States	Components	133	6743237.5339	1106209.8800000	0.100000	8	0.067974	0.000536
2020	2	Canada	Accessories	23	1518232.3586	13074.1000000	0.036827	8	0.069560	0.000509
2020	2	Canada	Bikes	63	3221990.8292	2586769.2289000	0.031017	8	0.068116	0.000587
2020	2	Canada	Clothing	46	2757494.7035	51598.5700000	0.053271	8	0.068456	0.000489

Query executed successfully. (local)\OLTP (16.0 RTM) DESKTOP-LEIQFVB\hp (53) AdventureWorks-OLTP 00:08:02 720 rows

(720 rows affected)

SQL Server Execution Times:
CPU time = 7044 ms, elapsed time = 475715 ms.

Completion time: 2025-07-16T16:45:08.2888627+03:00

Query executed successfully. (local)\OLTP (16.0 RTM) DESKTOP-LEIQFVB\hp (53) AdventureWorks-OLTP 00:08:02 720 rows

OLAP

Query.sql - (local)\OLAP.AdventureWorks-DW (DESKTOP-LEIQFVB\hp (58)) - Microsoft SQL Server Management Studio

```
USE [AdventureWorks-DW];
GO

-- Enable IO and Time statistics
SET STATISTICS TIME ON;

-- Show actual execution plan
-- (In SSMS press Ctrl+M or click "Include Actual Execution Plan")

WITH OrderTaxFreight AS (
    SELECT
        SalesOrderID,
```

Year	Month	Country	Category	OrderCount	TotalSales	GrossSales	AvgDiscountPerUnit	AvgDaysToShip	TaxRatio	FreightRatio
2020	1	Canada	Accessories	32	1949131.6500	23677.39	0.10	8	0.069221	0.000467
2020	1	Canada	Bikes	69	3180100.7985	2842903.9754	0.10	8	0.069727	0.000615
2020	1	Canada	Clothing	57	2829145.2912	70295.61	0.1049	8	0.069936	0.000566
2020	1	Canada	Components	54	2793489.4569	358905.73	0.10	8	0.069999	0.000579
2020	1	United Kingdom	Accessories	8	584493.8910	4390.46	0.10	8	0.076375	0.000446
2020	1	United Kingdom	Bikes	25	1122157.4419	971112.1327	0.10	8	0.073149	0.000653
2020	1	United Kingdom	Clothing	18	879673.6899	18486.49	0.10	8	0.074902	0.000630
2020	1	United Kingdom	Components	20	1033178.9728	163246.425	0.10	8	0.072475	0.000558
2020	1	United States	Accessories	68	4250450.8715	55974.09	0.10	8	0.066130	0.000455
2020	1	United States	Bikes	181	7525602.5123	6494776.4955	0.10	8	0.068157	0.000654
2020	1	United States	Clothing	131	6404687.0609	164990.50	0.1042	8	0.067914	0.000556
2020	1	United States	Components	133	6743237.5339	1106209.88	0.10	8	0.067974	0.000536
2020	2	Canada	Accessories	23	1518232.3586	13074.10	0.0368	8	0.069560	0.000509
2020	2	Canada	Bikes	63	3221990.8292	2586769.2289	0.031	8	0.068116	0.000587
2020	2	Canada	Clothing	46	2757494.7035	51598.57	0.0532	8	0.068456	0.000489

Query executed successfully. (local)\OLAP (16.0 RTM) DESKTOP-LEIQFVB\hp (58) AdventureWorks-DW 00:03:44

(720 rows affected)

SQL Server Execution Times:
CPU time = 3391 ms, elapsed time = 216614 ms.

Completion time: 2025-07-16T16:50:18.6014797+03:00

Query executed successfully. (local)\OLAP (16.0 RTM) DESKTOP-LEIQFVB\hp (58) AdventureWorks-DW 00:03:44 720 rows

Results

OLTP vs OLAP Queries

	OLTP	OLAP
Rows affected	720	720
Elapsed time	475716ms	216614ms
CPU time	7044 ms	3391ms

Insights:

- OLAP is ~2.5x faster in elapsed time for the same data.
- It uses less CPU, thanks to optimized schema and indexes.
- As data grows, the performance gap increases significantly:
 - OLTP struggles with joins & normalization.
 - OLAP thrives with denormalized star schema and columnstore indexes.

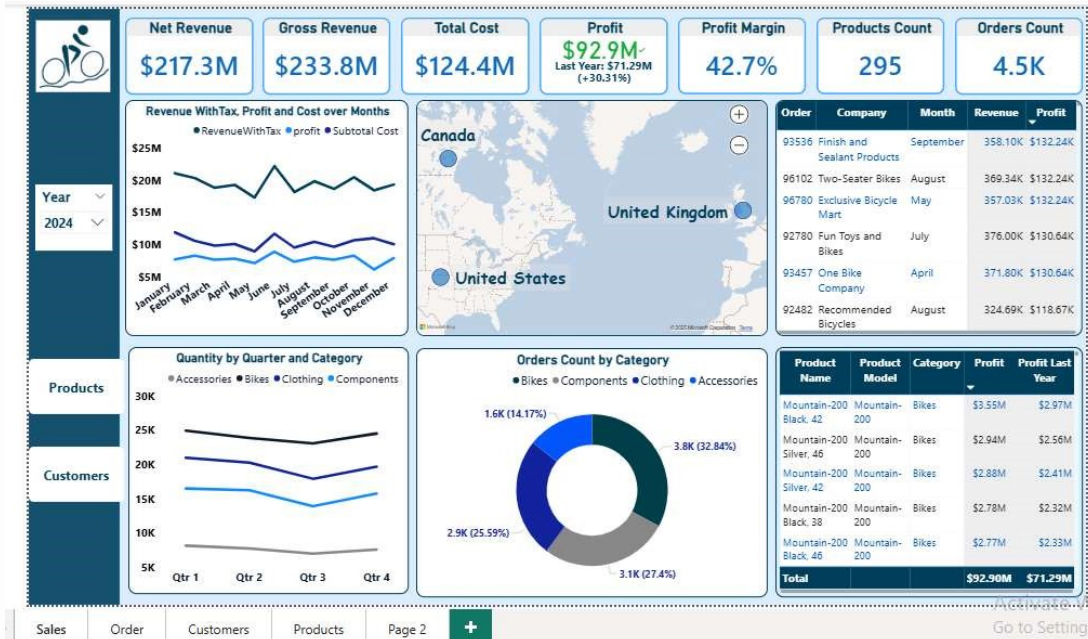
Power BI Sales & Customer Insights Dashboard

Turning Data into Actionable Insights

Introduction

- Purpose of the dashboard:
 - To monitor sales performance, profitability, and customer behavior.
- Scope:
 - Sales
 - Orders
 - Customers
 - Products.
- Data Coverage:
 - 2024 with **drill-through** analysis for deeper insights.

Sales Overview



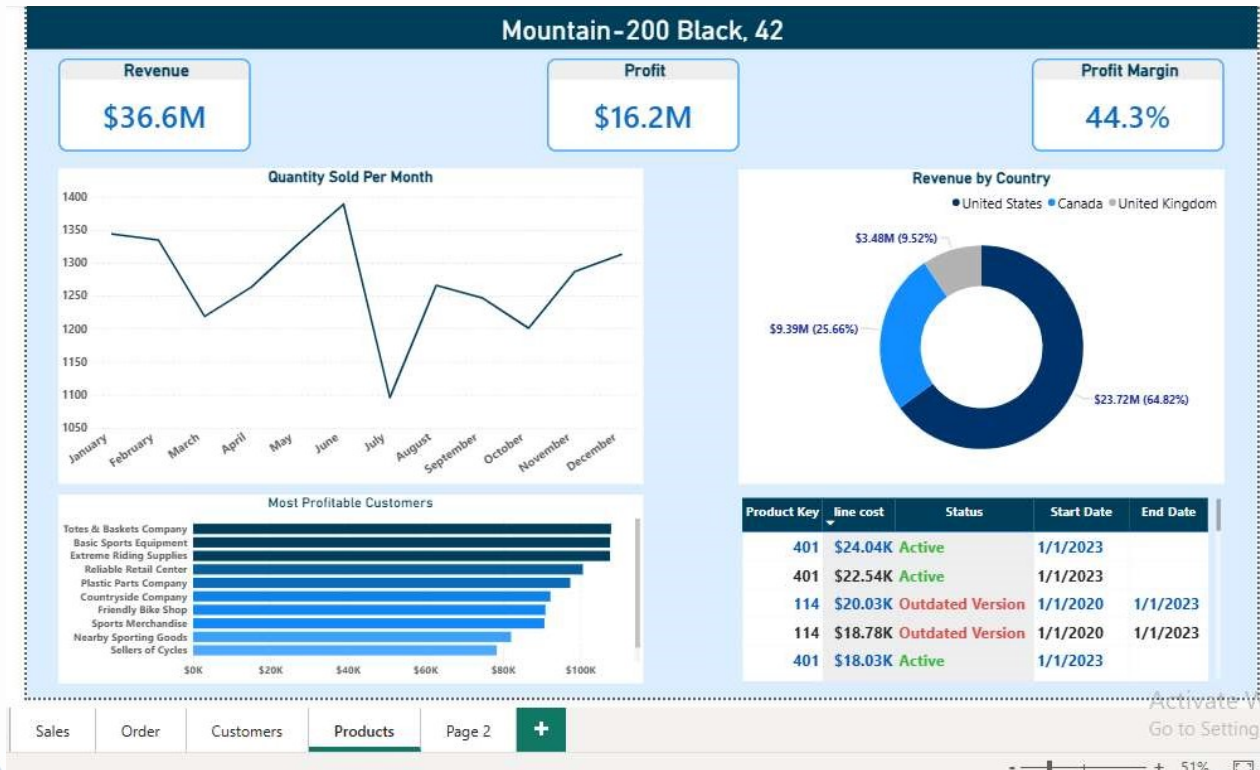
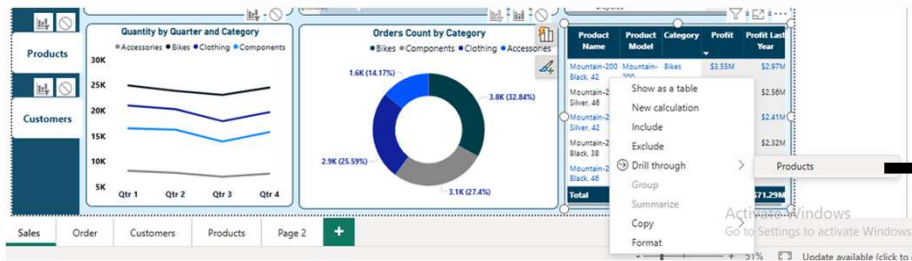
Sales Insights:

- Revenue growth is **outpacing** cost growth. Revenue with tax (\$233.8M) and profit (\$92.9M) show a healthy **42.7% profit margin**, indicating strong pricing power and operational efficiency.
- +30.3%** YoY profit increase shows effective cost control.
- Profit margin improvement driven mainly by bike sales (most ordered).
- Plastic Parts Company from Canada is the **most profitable** customer.
- Peak revenues** at January & June.
- Customer base optimization: 40% of customers are **inactive**, representing untapped potential that can be converted into revenue through targeted reactivation campaigns.
- Customer concentration risk — A few clients (e.g., Totes & Baskets, Basic Sports Equipment) contribute a large share of profits, which could be risky if they churn.
- Average shipping time of 6 days is competitive but may not be optimal for high-value customers.

Customer:



Product-specific Analysis



- The **United States** is the dominant revenue driver across several product lines, with the Mountain-200 Black model reaching a **64.8%** share in the US market. Heavy reliance on one region poses risks if market conditions shift, so expanding presence in the UK and Canada is essential to diversify and balance revenue streams.
- The product-specific segment is highly profitable with a 44.3% margin, reflecting strong cost control and market competitiveness.
- sales **peaking** in June and dipping in July.

Customer-specific Analysis



A screenshot of a Power BI context menu. The menu is open, showing options like 'Show as a table', 'New calculation', 'Include', 'Exclude', 'Drill through', 'Group', 'Summarize', 'Copy', and 'Format'. The 'Drill through' option is highlighted, and a sub-menu is visible showing 'Customers' as the selected drill-through target.

---> using Drill-through



- Revenue is heavily concentrated in a few high-demand products, suggesting opportunities to expand or replicate successful product lines.
- Seasonal trends indicate demand peaks during specific months, likely tied to production cycles, customer procurement schedules, or industry-specific events.

Order-specific Analysis

Order	Company	Month	Revenue	Profit
93536	Finish and	September	358.10K	\$132.24K
961			9.34K	\$132.24K
967			7.03K	\$132.24K
927			6.00K	\$130.64K

using Drill-through



Expansion Potential

1. Capitalize on the loyal bike customer base by promoting accessories such as helmets and gear, which usually offer higher profit margins.
2. Customer Reactivation: Focus on re-engaging the inactive 40% of customers with personalized offers, prioritizing underperforming regions to boost sales.
3. Cost optimization: Focus on top-selling low-margin products to identify cost-reduction opportunities without sacrificing quality.
4. Geographic diversification: Reduce US dependency by targeted marketing in UK and Canada.
5. Shipping Efficiency: Reducing shipping time could increase customer satisfaction and repeat purchases.

THANK YOU!



Ahmed Samy



a7madmohsen.10@gmail.com